

Join the discord : <https://discord.gg/AE6JG7xnK>

Access the online doc : <https://4hands2cats.github.io/UnityDebugToolkitDocumentation/EGT/#/>

ESCAPE GAME TOOLKIT DOCUMENTATION

EGT (Escape game toolkit)

EGT (Escape Game Toolkit)

EGT is an upcoming toolkit designed to make creating escape games as easy and intuitive as possible. It's currently in development and will launch as a full-featured asset upon release.

Modular Approach

We're developing individual features as separate assets. Once everything is ready, we'll combine them into a single, comprehensive toolkit.

Bundle Discount

If you purchase any individual module, you'll receive a discount on the full pack when it's released.

Support Development

You can support our progress by getting one of the smaller toolkits!

Current state of the toolkit

 **Next to be released : The Inventory Toolkit**  85% Ready

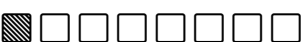
The Inventory Toolkit is the first release in the EGT pack. It gives you powerful tools to build flexible, intuitive inventory systems for your escape game.

 Join our Discord to request features, share ideas, or give feedback. Your input helps shape the future of the toolkit!

 Not released yet : The Settings Menu Toolkit  65% Ready

 Not released yet : The Sprite maker tool  50% Ready

 Not released yet : The status system  30% Ready

 Not released yet : The Dialogue system  10% Ready

 Not released yet : The Quest System  10% Ready

✖ Not release yet : The Scenario System ☐☐☐☐☐☐☐☐☐☐☐ 0% Ready

More system are going to be added

The Inventory Toolkit is a powerful, modular system built to handle all your in-game item management needs with ease and flexibility.

✦ Key Features

🗪 Modular Architecture

Designed with modularity in mind, allowing you to easily add, replace, or extend components without rewriting the core.

Grid-Based Inventory

Includes a flexible, drag-and-drop grid system with support for different item sizes, stacking, and sorting.

Hotbar Hotbar

Comes with a fully functional hotbar that can be tailored to suit quick-access needs, such as weapons or tools.

Save & Load System

Persistent inventory state using a built-in save system. Inventory, item positions, items amount and equipped item are saved between sessions

Interaction system

It allows you to use, pick and grab items. It easy to expand and fully modular (completely independant from the Inventory system). Look at the documentation for the interaction system for more informations

🕒 What's Next?

In the next few sections, we'll guide you through the essentials of getting started with the Inventory Toolkit:

🎮 Demo Scene Setup

Learn how to get the demo scene up and running, so you can see the system in action.

🔗 Player Manager Overview

We'll break down the key components in the Player Manager's inspector, helping you understand what's what and how to tweak things.

🖥 Under the Hood

Finally, we'll dive into the Player Manager's code to explore how it works—and how you can extend it.

🗪 Architecture Overview

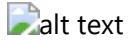
The Inventory Toolkit is built on a clean and modular architecture composed of three main layers:

- UI
- Model
- Data

Each layer is decoupled and communicates using Data Transfer Objects (DTOs). This design allows for easy replacement and customization of parts like the save system, which is non-invasive and fully modular. If you're considering integrating your own save solution and aren't sure where to start, feel free to reach out on our Discord server!

The interaction system connects to the PlayerManager, which acts as a central relay between the player's actions and the inventory system.

The UI and Model are fully decoupled using a ScriptableObject-based presenter, following the Model–View–Presenter (MVP) pattern.



How to Enable the Inventory System

Check out the demo scene included in the package. You'll notice two key objects in the hierarchy:

- PlayerManager
- UI

Try It Out (Demo Scene Walkthrough)

Here's a simple demo scenario to get started:

Run the scene your character should be able to move around (based on a slightly modified third-person controller).

1. Try opening the door.
2. Pick up the lamp on the left side.
3. Press I to open your inventory.
4. Drag and drop the lamp into the hotbar.
5. Use number keys (e.g. 1, 2, 3, ...) to equip the lamp.
6. With the lamp equipped, use it to find the key on the right.
7. Equip the key from your inventory.
8. Walk to the door again with the key equipped—and open it.

 Congrats! You've opened the door.

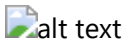
Feel free to explore the scene! Each interactable item has its own logic through custom interaction components. Right now, there are only two built-in types, but we're actively expanding the toolkit.

 Want to see more interaction types? Join the Discord and share your ideas!

The Player Manager is the main component of the toolkit. It's a MonoBehaviour that acts as the central hub between the UI, the inventory model, the save system, and the interaction logic.

Inspector overview

From this inspector, you can configure and control everything related to your inventory setup:



From top to bottom, you'll find:

- A reference to the inventory prefab (used to spawn the inventory logic).
 - The UI Presenter ScriptableObject, which handles communication between the UI and the model.
- A section for input keys — used to control the hotbar and inventory actions.
- A list of starting items to pre-populate the inventory at runtime.
- A reference for where items are dropped in the world when removed.
- The Interactor, which handles in-world interactions (like picking up items).
- The Save System, with quick save/load buttons for testing.

Everything here should already be set up if you're using the demo scene. If you're adding this to your own setup, just drag in the proper references and you're good to go.



Code overview

We are just going to focus on the player manager. If you need more information about the rest of the code base do not hesitate to request some more documentation or help in the discord server.

So on awake it initializes the save system and binds to the the interaction system.

```
private void Awake()
{
    interactor.OnInteract += Interactor_OnInteract;
    saveLoadSystem.Init();
    saveLoadSystem.NewGameFile();
}
```

Each time the interactor interacts it handles the different kinds of interactions. Notice that those interactions are notifying the Player manager, but the are also doing things on their ends.

```
private void Interactor_OnInteract(IInteractable obj, Action interaction)
{
    switch (obj)
    {
        case GrabItemInteractable grabItemInteractable :
            if (grabItemInteractable.Payload is IItem item)
            {
                _gridInventory.AddItem(item, interaction, null);
            }
            break;
        case UseItemInteractable useItemInteractable:
            if (useItemInteractable.Payload is IItem itemToUse)
            {
                var activeItem =
                _hotBarInventory.CheckIfIsActiveItem(itemToUse);
                if (activeItem != null)
            }
    }
}
```

```

        {
            interaction?.Invoke();
            if (useItemInteractable.ConsumeItem)
            {
                _gridInventory.RemoveItem(activeItem, null, null);
            }
        }
    }
    break;
}
}
}

```

On the start we Initialize everything using all the data set up in the inspector. The inventories are not mono behaviours. You can add as many instances as you wish. You are not forced to have a user interface to store items in your UI. You can use the base class Inventory if you want a vanilla data structure that is handled by the save system already.

There is a dependency system. If you add a dependency to an inventory it means that every changes will be cascade to their dependency. Don't make cyclic dependency.

```

private async void Start()
{
    _gridInventory = new
GridInventory(gridInventoryData.InventoryPresenter,
gridInventoryData.InventorySize, gridInventoryData.ItemsTypes);
    _gridInventory.OnItemDrop += GridInventory_OnItemDrop;

    await Awaitable.NextFrameAsync();

    for(int i = 0; i< startingItems.Count; i++)
    {
        _gridInventory.AddItem(startingItems[i].Item, null, null);
    }

    _hotBarInventory = new
HotBarInventory(hotBarInventoryData.InventoryPresenter,
hotBarInventoryData.InventorySize.x, hotBarInventoryData.ItemsTypes);
    hotBarInputsHandler.Init(_hotBarInventory);

    _gridInventory.AddDependency(_hotBarInventory);
}

```

Now only the binding with the save system left to do. There are two methods : Save and Load. The save system uses the principle of the data binding. GameData, InventoryData, etc ... are only known from the save system and the Player Manager. The reste of the system uses Dto. Its a bit of a duplicated data behaviour but it also leave you with the possibility of using your save system of choice if you wish.

Saving is really easy : one just binds the dtos of the inventory instances to the save system. Now everytime there is a change the save system knows about it. It saves a load on saving performance.

Don't forget to call save to save when needed.

```
private void SaveInventory()
{
    SaveLoadSystem.Instance.BindDto(_gridInventory.GetInventoryDto());
    SaveLoadSystem.Instance.BindDto(_hotBarInventory.GetInventoryDto());
}
```

Loading is tougher : there is an order to respect You need to load your inventories in there dependency order. The grid depends on the hot bar. So first load the grid and then the hotbar. The system binds the dtos of the inventories once more.

```
private void LoadInventory()
{
    var data = SaveLoadSystem.Instance.LoadGame("NewGame");
    var gridInventoryDto = data.GetInventoryOfType(EInventoryType.Grid);
    if (gridInventoryDto != null)
    {
        foreach (var itemDto in gridInventoryDto.Items)
        {
            itemLibrary.MatchAndUpdateItemDtoBySO_ID(itemDto);
        }
        _gridInventory.LoadFromDto(gridInventoryDto);
    }
    else
    {
        Debug.Log("No Grid Inventory found in saved data.");
    }

    // Loaded after the grid because the grid control the hotbar in the
    // current set up
    // You can change the dependencies, or create new ones but beware of
    // the loading order
    var hotBarInventoryDto = data.GetInventoryOfType(EInventoryType.HotBar,
gridInventoryDto.Items);
    if (hotBarInventoryDto != null)
    {
        _hotBarInventory.LoadFromDto(hotBarInventoryDto);
    }
    else
    {
        Debug.Log("No Hotbar Inventory found in saved data.");
    }
}
```

In the next section you'll learn about how to create new items for your project.

Items in the system are defined using the ItemSO Scriptable Object. You can find some sample items in: `Assets/4Hands2Cats/EscapeGameToolkit/Inventory/Items/Sample-data`

Creating a New Item

To create a new item, right-click in the Project window and go to:

Create > EGT > Inventory > Items > BaseSOItem

Item Setup

To be compatible with the save/load system, all items must be added to the Item Library.

Each item can have a custom size and shape, which defines how it fits within the grid-based inventory. By choosing Custom mode, you can design items with any shape, perfect for Tetris-like inventory behavior.

You can also specify whether the item is stackable enabling the inventory system to manage multiple quantities in a single slot.

Finally, assign a type and a prefab to represent the item visually in 2D or 3D.

To make the item interactive, add an interaction component (like the GrabInteraction) to the prefab.